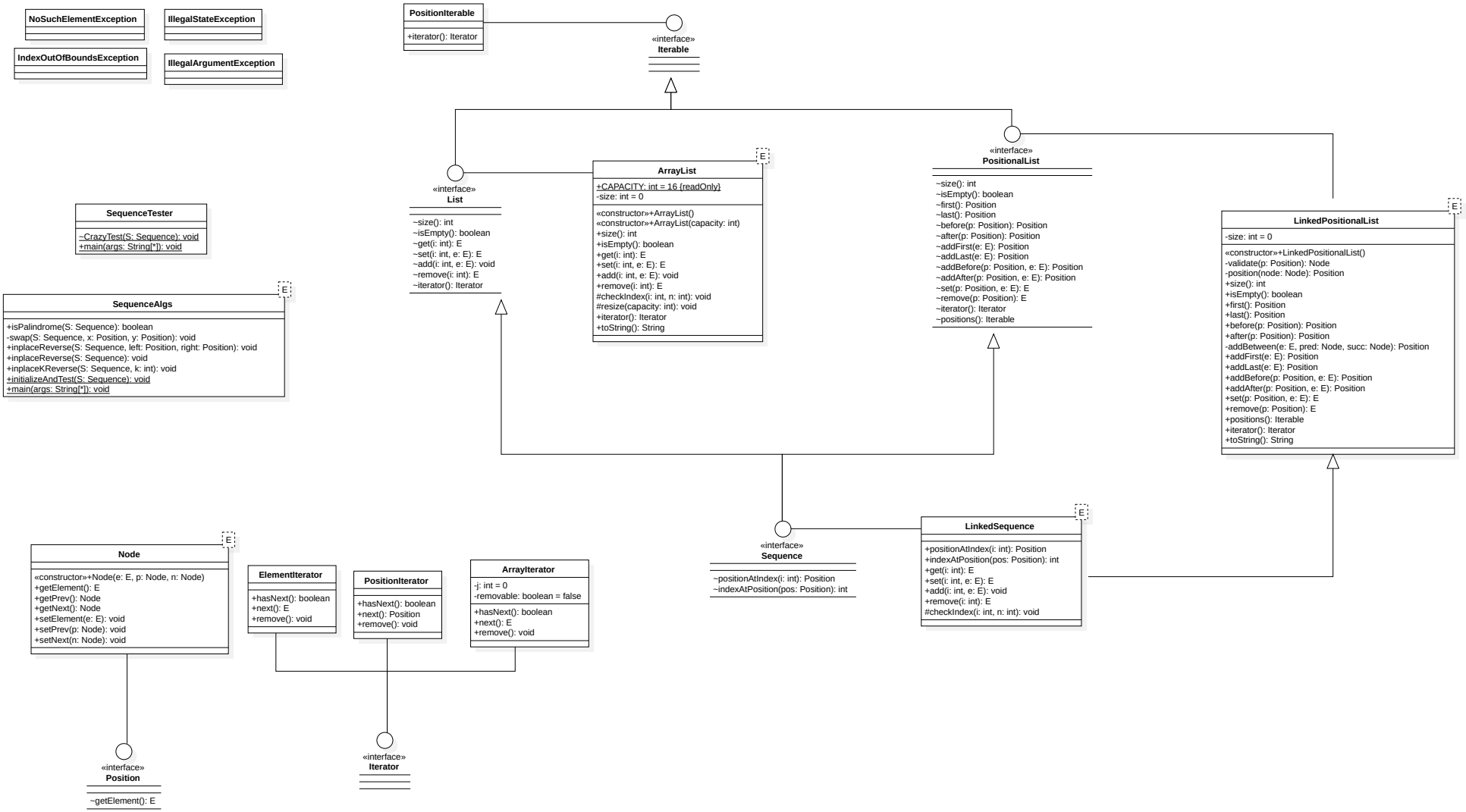
NoSuchElementException
IllegalStateException
IndexOutOfBoundsException
IllegalArgumentException
PositionIterable
+iterator(): Iterator
«interface»
Iterable
«interface»
List
~size(): int
~isEmpty(): boolean
~get(i: int): E
~set(i: int, e: E): E
~add(i: int, e: E): void
~remove(i: int): E
~iterator(): Iterator
ArrayList
E
+CAPACITY: int = 16 {readOnly}
-size: int = 0
«constructor»+ArrayList()
«constructor»+ArrayList(capacity: int)
+size(): int
+isEmpty(): boolean
+get(i: int): E
+set(i: int, e: E): E
+add(i: int, e: E): void
+remove(i: int): E
#checkIndex(i: int, n: int): void
#resize(capacity: int): void
+iterator(): Iterator
+toString(): String
«interface»
PositionalList
~size(): int
~isEmpty(): boolean
~first(): Position
~last(): Position
~before(p: Position): Position
~after(p: Position): Position
~addFirst(e: E): Position
~addLast(e: E): Position
~addBefore(p: Position, e: E): Position
~addAfter(p: Position, e: E): Position
~set(p: Position, e: E): E
~remove(p: Position): E
~iterator(): Iterator
~positions(): Iterable
LinkedPositionalList
E
-size: int = 0
«constructor»+LinkedPositionalList()
-validate(p: Position): Node
-position(node: Node): Position
+size(): int
+isEmpty(): boolean
+first(): Position
+last(): Position
+before(p: Position): Position
+after(p: Position): Position
-addBetween(e: E, pred: Node, succ: Node): Position
+addFirst(e: E): Position
+addLast(e: E): Position
+addBefore(p: Position, e: E): Position
+addAfter(p: Position, e: E): Position
+set(p: Position, e: E): E
+remove(p: Position): E
+positions(): Iterable
+iterator(): Iterator
+toString(): String
SequenceTester
~CrazyTest(S: Sequence): void
+main(args: String[*]): void
SequenceAlgs
E
+isPalindrome(S: Sequence): boolean
-swap(S: Sequence, x: Position, y: Position): void
+inplaceReverse(S: Sequence, left: Position, right: Position): void
+inplaceReverse(S: Sequence): void
+inplaceKReverse(S: Sequence, k: int): void
+initializeAndTest(S: Sequence): void
+main(args: String[*]): void
«interface»
Sequence
~positionAtIndex(i: int): Position
~indexAtPosition(pos: Position): int
LinkedSequence
E
+positionAtIndex(i: int): Position
+indexAtPosition(pos: Position): int
+get(i: int): E
+set(i: int, e: E): E
+add(i: int, e: E): void
+remove(i: int): E
#checkIndex(i: int, n: int): void
Node
E
«constructor»+Node(e: E, p: Node, n: Node)
+getElement(): E
+getPrev(): Node
+getNext(): Node
+setElement(e: E): void
+setPrev(p: Node): void
+setNext(n: Node): void
ElementIterator
+hasNext(): boolean
+next(): E
+remove(): void
PositionIterator
+hasNext(): boolean
+next(): Position
+remove(): void
ArrayIterator
-j: int = 0
-removable: boolean = false
+hasNext(): boolean
+next(): E
+remove(): void
«interface»
Position
~getElement(): E
«interface»
Iterator